

用于一步生成建模的平均流

Zhengyang Geng^{1*} Mingyang Deng² Xingjian Bai² J. Zico Kolter¹ Kaiming He²

¹ 卡内基梅隆大学

² 麻省理工学院

摘要

本文提出一个原则性强且高效的一步生成建模框架。我们引入平均速度的概念来刻画流场，与流匹配方法所建模的瞬时速度形成对比。我们推导出平均速度与瞬时速度之间的一个良定义恒等式，并利用该恒等式指导神经网络训练。本文方法称为平均流模型，具有自包含性，无需预训练、蒸馏或课程学习。平均流模型展现出强大的经验性能：在从零开始训练的 ImageNet 256×256 上，仅需一次函数评估（1-NFE）即可达到 3.43 的 FID，显著优于此前最先进的一步扩散/流模型。本研究大幅缩小了一步扩散/流模型与其多步前身之间的差距，我们希望它能激励未来研究重新审视这些强大模型的基础。

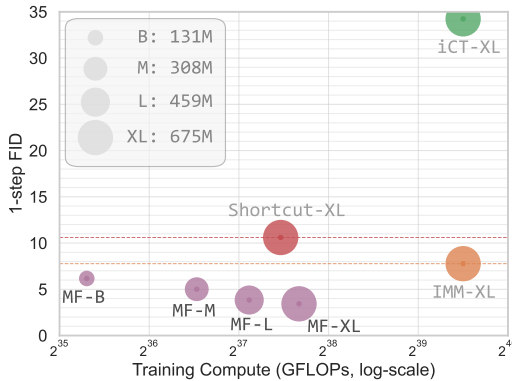


图 1：从零开始在 ImageNet 256×256 上的一步生成。我们的平均流（MF）模型相比此前最先进的一步扩散/流方法，生成质量显著更优。此处 iCT [43]、Shortcut [13] 和我们的 MF 均为 1-NFE 生成，而 IMM 的一步结果 [52] 涉及 2-NFE 引导。详细数值见表 2。所示图像由我们的 1-NFE 模型生成。

1 引言

生成建模的目标是将先验分布变换为数据分布。流匹配 [28, 2, 30] 提供了一个直观且概念简单的框架，用于构建将一个分布传输到另一个分布的流路径。流匹配与扩散模型 [42, 44, 19] 密切相关，其关注点在于指导模型训练的速度场。自提出以来，流匹配已在现代生成建模中得到广泛应用 [11, 33, 35]。

流匹配（Flow Matching）和扩散模型在生成过程中都采用迭代采样。近期研究对少步——尤其是一步前馈——生成模型给予了显著关注。作为该方向的开创性工作，一致性模型（Consistency Models）[46, 43, 15, 31] 对沿同一路径采样的输入施加网络输出的一致性约束。尽管结果令人鼓舞，但一致性约束仅作为网络行为的属性被施加，而应指导学习的底层真实场的性质仍属未知。因此，训练可能不稳定，且需要精心设计的“离散化课程”[46, 43, 15] 来逐步约束时间域。* 部分工作于访问麻省理工学院期间完成。

本文提出一个原则性强且有效的框架，称为平均流（MeanFlow），用于一步生成。核心思想是引入一个新的真实场来表示平均速度，与流匹配中通常建模的瞬时速度形成对比。平均速度定义为位移与时间间隔之比，其中位移由瞬时速度的时间积分给出。仅从该定义出发，我们推导出平均速度与瞬时速度之间定义明确的内在关系，这自然成为指导网络训练的原则性基础。

基于这一基本概念，我们训练一个神经网络直接建模平均速度场。我们引入一个损失函数，促使网络满足平均速度与瞬时速度之间的内在关系。无需额外的一致性启发式。真实目标场的存在确保最优解原则上独立于具体网络，这在实际中可带来更稳健和稳定的训练。我们进一步表明，该框架可自然地将无分类器引导（Classifier-Free Guidance, CFG）[18] 纳入目标场，在采样时使用引导不会产生额外成本。

我们的平均流模型在一步生成建模中展现出强大的实证性能。在 ImageNet 256×256 [7] 上，我们的方法使用 1-NFE（函数评估次数）生成达到了 3.43 的 FID。这一结果显著优于同类先前最先进的方法，相对提升幅度达 50% 至 70%（图 1）。此外，我们的方法是一个自包含的生成模型：完全从零开始训练，无需任何预训练、蒸馏或课程学习。我们的研究在很大程度上缩小了一步扩散/流模型与其多步前身之间的差距，我们希望它能启发未来工作重新审视这些强大模型的基础。

2 相关工作

扩散与流匹配。在过去十年中，扩散模型 [42, 44, 19, 45] 已发展成为一个非常成功的生成建模框架。这些模型逐步向干净数据添加噪声，并训练神经网络来逆转这一过程。该过程涉及求解随机微分方程（SDE），随后被重新表述为概率流常微分方程（ODE）[45, 22]。流匹配方法 [28, 2, 30] 通过建模定义分布之间流动路径的速度场来扩展这一框架。流匹配也可以被视为连续时间归一化流 [36] 的一种形式。

少步扩散/流模型。减少采样步骤已成为实践和理论两方面的重要考量。一种方法是将预训练的多步扩散模型蒸馏为少步模型，例如 [39, 14, 41] 或分数蒸馏 [32, 50, 53]。早期对训练少步模型的探索 [46] 建立在基于蒸馏的方法的演进之上。与此同时，一致性模型 [46] 被开发为一种独立的生成模型，无需蒸馏。这些模型对网络在不同时间步的输出施加一致性约束，鼓励它们沿轨迹产生相同的端点。各种一致性模型和训练策略 [46, 43, 15, 31, 49] 已被研究。

在近期工作中，多种方法聚焦于刻画基于扩散或流的量相对于两个时间相关变量的特性。在 [3] 中，流映射被定义为两个时间步之间流的积分，并提出了多种匹配损失形式用于学习。与本文方法所基于的平均速度相比，流映射对应于位移。捷径模型 [13] 在流匹配之外引入自洽性损失函数，捕捉不同离散时间间隔上流之间的关系。归纳矩匹配 [52] 对不同时间步上随机插值的自洽性进行建模。

3 背景：流匹配

流匹配 [28, 30, 1] 是一类生成模型，学习匹配两个概率分布之间的流，流由速度场表示。形式上，给定数据 $x \sim p_{\text{data}}(x)$ 和先验 $\epsilon \sim p_{\text{prior}}(\epsilon)$ ，可构造流路径 $z_t = a_t x + b_t \epsilon$ ，其中时间 t ， a_t 和 b_t 为预定义调度。速度 v_t 定义为 $v_t = \dot{z}_t = a'_t x + b'_t \epsilon$ ，其中 $'$ 表示时间导数。该速度在 [28] 中被称为条件速度，记为 $v_t = v_t(z_t | x)$ ，见图 2 左。常用调度为 $a_t = 1 - t$ 和 $b_t = t$ ，由此得到 $v_t = \epsilon - x$ 。

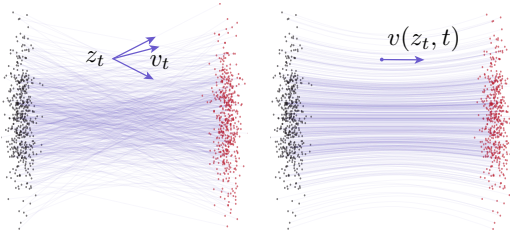


图 2：流匹配中的速度场 [28]。左：条件流 [28]。给定的 z_t 可能来自不同的 (x, ϵ) 对，导致不同的条件速度 v_t 。右：边缘流 [28]，通过对所有可能的条件速度进行边缘化得到。边缘速度场作为网络训练的真实底层场。此处所示所有速度本质上均为瞬时速度。图示参考 [12]。（灰点：先验样本；红点：数据样本。）

由于给定的 z_t 及其 v_t 可能来自不同的 x 和 ϵ ，流匹配本质上对所有可能性求期望，称为边缘速度 [28]（图 2 右）：

$$v(z_t, t) \triangleq \mathbb{E}_{p_t(v_t|z_t)}[v_t]. \quad (1)$$

由参数 θ 化的神经网络 v_θ 被学习以拟合边缘速度场： $\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, p_t(z_t)} \|v_\theta(z_t, t) - v(z_t, t)\|^2$ 。式 (1) 中的边缘化，计算该损失函数不可行，但提出改为评估条件流匹配损失

[28]: $\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, x, \epsilon} \|v_\theta(z_t, t) - v_t(z_t | x)\|^2$, where the target v_t is the conditional velocity. Minimizing $\mathcal{L}_{\text{CFM}}$ is equivalent to minimizing $\mathcal{L}_{\text{FM}}$ [28].

Given a marginal velocity field $v(z_t, t)$, samples are generated by solving an ODE for z_t :

$$\frac{d}{dt} z_t = v(z_t, t) \quad (2)$$

从 $z_1 = \epsilon \sim p_{\text{prior}}$ 开始。解可写为： $z_r = z_t - \int_r^t v(z_\tau, \tau) d\tau$ ，其中用 r 表示另一个时间步。

实际中，该积分在离散时间步上数值近似。例如，欧拉方法作为一阶常微分方程求解器，每一步计算为： $z_{t_{i+1}} = z_{t_i} + (t_{i+1} - t_i)v(z_{t_i}, t_i)$ 。也可应用高阶求解器。

值得注意的是，即使条件流被设计为直线（“校正”）[28, 30]，边缘速度场（公式 (1)）通常也会诱导出弯曲的轨迹。参见图 2 的说明。我们还强调，这种非线性不仅是神经网络近似的结果，而是源于底层真实的边缘速度场。当在弯曲轨迹上应用粗离散化时，数值常微分方程求解器会导致结果不准确。

4 平均流模型

4.1 平均流

我们方法的核心思想是引入一个表示平均速度的新场，而流匹配中建模的速度表示瞬时速度。

平均速度。 我们将平均速度定义为两个时间步 t 和 r 之间的位移（通过积分获得）除以时间间隔。形式上，平均速度 u 为：

$$u(z_t, r, t) \triangleq \frac{1}{t-r} \int_r^t v(z_\tau, \tau) d\tau. \quad (3)$$

为了强调概念上的差异，在本文中，我们使用符号 u 表示平均速度，使用 v 表示瞬时速度。

$u(z_t, r, t)$ 是一个联合依赖于 (r, t) 的场。 u 的场如图 3 所示。注意，一般来说，平均速度 u 是瞬时速度 v 的泛函的结果：即， $u = \mathcal{F}[v] \triangleq \frac{1}{t-r} \int_r^t v d\tau$ 。它是一个由 v 诱导的场，不依赖于任何神经网络。从概念上讲，正如瞬时速度 v 在流匹配中作为真实场一样，平均速度 u 在我们的公式中为学习提供了一个底层的真实场。

根据定义， u 的场满足某些边界条件和“一致性”约束（推广了 [46] 的术语）。当 $r \rightarrow t$ 时，我们有： $\lim_{r \rightarrow t} u = v$ 。此外，一种“一致性”形式自然满足：对于任何中间时间 s ，在 $[r, t]$ 上迈出较大的一步与在 $[r, s]$ 和 $[s, t]$ 上迈出两个较小的连续步骤是“一致的”。要看到这一点，观察到 $(t-r)u(z_t, r, t) = (s-r)u(z_s, r, s) + (t-s)u(z_t, s, t)$ ，这直接由可加性得出。

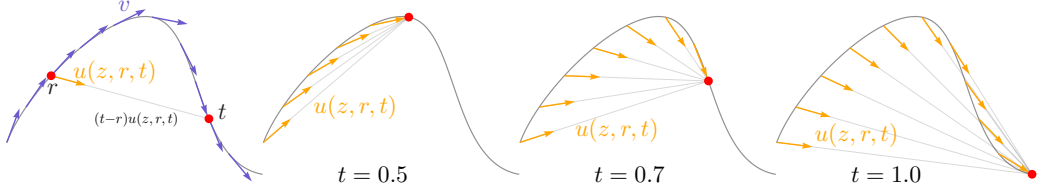


图 3: 平均速度场 $u(z, r, t)$. 最左侧: 瞬时速度 v 决定路径的切线方向, 而由方程 (3) 定义的平均速度 $u(z, r, t)$ 通常不与 v 对齐。平均速度与位移对齐, 位移为 $(t - r)u(z, r, t)$ 。右侧三个子图: 场 $u(z, r, t)$ 同时以 r 和 t 为条件, 此处展示的是 $t = 0.5, 0.7$ 和 1.0 的情况。

积分的: $\int_r^t v d\tau = \int_r^s v d\tau + \int_s^t v d\tau$ 。因此, 一个能准确逼近真实 u 的网络, 预期会自然满足一致性关系, 无需显式约束。

本文 MeanFlow 模型的最终目标是用神经网络 $u_\theta(z_t, r, t)$ 逼近平均速度。这一方法具有显著优势: 假设我们能准确逼近该量, 则仅需一次 $u_\theta(\epsilon, 0, 1)$ 的评估即可逼近整条流路径。换言之, 正如本文将通过实验证明的, 该方法更适合单步或少步生成, 因为在推理时无需显式逼近时间积分, 而建模瞬时速度时则需要这样做。然而, 直接使用方程 (3) 定义的平均速度作为训练网络的真值是不可行的, 因为训练过程中需要计算积分。本文的关键洞见在于, 平均速度的定义方程可以经过变换, 构造出一个最终适合训练的优化目标, 即使只能获取瞬时速度。

MeanFlow 恒等式。 为了得到适合训练的公式, 本文将方程 (3) 改写为:

$$(t - r)u(z_t, r, t) = \int_r^t v(z_\tau, \tau) d\tau. \quad (4)$$

现在对两边关于 t 求导, 将 r 视为与 t 无关。得到:

$$\frac{d}{dt}(t - r)u(z_t, r, t) = \frac{d}{dt} \int_r^t v(z_\tau, \tau) d\tau \implies u(z_t, r, t) + (t - r) \frac{d}{dt} u(z_t, r, t) = v(z_t, t), \quad (5)$$

其中左侧的变换使用了乘积法则, 右侧使用了微积分基本定理²。整理各项后, 得到恒等式:

$$\underbrace{u(z_t, r, t)}_{\text{average vel.}} = \underbrace{v(z_t, t)}_{\text{instant. vel.}} - \underbrace{(t - r) \frac{d}{dt} u(z_t, r, t)}_{\text{time derivative}} \quad (6)$$

本文将这个方程称为“MeanFlow 恒等式”, 它描述了 v 与 u 之间的关系。容易证明方程 (6) 与方程 (4) 等价 (见附录 B.3)。

式 (6) 的右侧为 $u(z_t, r, t)$ 提供了一个“目标”形式, 我们将利用它构建损失函数来训练神经网络。为了作为合适的目标, 我们还必须进一步分解时间导数项, 这将在下文讨论。

计算时间导数。 为了计算式 (6) 中的 $\frac{d}{dt} u$ 项, 注意到 $\frac{d}{dt}$ 表示全导数, 它可以 $\frac{d}{dt}$ 展开为偏导数:

$$\frac{d}{dt} u(z_t, r, t) = \frac{dz_t}{dt} \partial_z u + \frac{dr}{dt} \partial_r u + \frac{dt}{dt} \partial_t u. \quad (7)$$

含 $\frac{dz_t}{dt} = v(z_t, t)$ (参见式 (2)), $\frac{dr}{dt} = 0$, and $\frac{dt}{dt} = 1$, 我们得到 u 与 v 之间的另一个关系:

$$\frac{d}{dt} u(z_t, r, t) = v(z_t, t) \partial_z u + \partial_t u, \quad (8)$$

若 r 依赖于 t , 莱布尼茨法则 [26] 给出: $\frac{d}{dt} \int_r^t v(z_\tau, \tau) d\tau = v(z_t, t) - v(z_r, r) \frac{dr}{dt}$.

该方程表明，全导数由函数 u 的雅可比矩阵 $[\partial_z u, \partial_r u, \partial_t u]$ 与切向量 $[v, 0, 1]$ 之间的雅可比-向量积 (JVP) 给出。在现代库中，这可以通过 jvp 接口高效计算，例如 PyTorch 中的 `torch.func.jvp` 或 JAX 中的 `jax.jvp`，我们稍后将讨论。

使用平均速度进行训练。 至此，公式与任何网络参数化无关。现在引入一个模型来学习 u 。形式上，将网络参数化为 u_θ ，并鼓励其满足平均流恒等式 (方程 (6))。具体地，最小化以下目标：

$$\mathcal{L}(\theta) = \mathbb{E} \|u_\theta(z_t, r, t) - \text{sg}(u_{\text{tgt}})\|_2^2, \quad (9)$$

$$\text{where } u_{\text{tgt}} = v(z_t, t) - (t - r)(v(z_t, t)\partial_z u_\theta + \partial_t u_\theta), \quad (10)$$

项 u_{tgt} 作为有效的回归目标，由方程 (6) 驱动。该目标仅使用瞬时速度 v 作为唯一的真实信号；无需积分计算。虽然目标应涉及 u 的导数 (即 ∂u)，但它们被其参数化对应物 (即 ∂u_θ) 所替代。在损失函数中，对目标 u_{tgt} 应用停止梯度 (sg) 操作，遵循常见做法 [46, 43, 15, 31, 13]：在我们的情况下，它消除了通过雅可比-向量积进行“双重反向传播”的需要，从而避免了高阶优化。尽管有这些可优化性实践，如果 u_θ 达到零损失，很容易证明它将满足平均流恒等式 (方程 (6))，从而满足原始定义 (方程 (3))。

方程 (10) 中的速度 $v(z_t, t)$ 是流匹配 [28] 中的边际速度 (见图 2 右)。我们遵循 [28] 将其替换为条件速度 (图 2 左)。由此，目标为：

$$u_{\text{tgt}} = v_t - (t - r)(v_t \partial_z u_\theta + \partial_t u_\theta). \quad (11)$$

回顾 $v_t = a'_t x + b'_t \epsilon$ 是条件速度 [28]，默认情况下， $v_t = \epsilon - x$ 。

最小化损失函数方程 (9) 的伪代码在算法 1 中给出。总体而言，我们的方法概念简单：其行为类似于流匹配，关键区别在于匹配目标被 $-(t - r)(v_t \partial_z u_\theta + \partial_t u_\theta)$ 修改，这源于我们对平均速度的考虑。特别地，注意如果限制条件 $t = r$ ，则第二项消失，该方法将完全匹配标准流匹配。

在算法 1 中，雅可比向量积运算非常高效。本质上，通过雅可比向量积计算 $\frac{d}{dt} u$ 仅需一次反向传播，类似于神经网络中的标准反向传播。由于 $\frac{d}{dt} u$ 是目标 u_{tgt} 的一部分，因此受到停止梯度 (w.r.t. θ) 的约束，神经网络优化 (w.r.t. θ) 的反向传播将 $\frac{d}{dt} u$ 视为常数，不会产生高阶梯度计算。因此，雅可比向量积仅引入一次额外的反向传播，其成本与反向传播相当。在我们对算法 1 的 JAX 实现中，开销不到总训练时间的 20% (见附录)。

采样。 使用平均流模型进行采样只需将时间积分替换为平均速度：

$$z_r = z_t - (t - r)u(z_t, r, t) \quad (12)$$

在一步采样的情况下，我们只需 $z_0 = z_1 - u(z_1, 0, 1)$, where $z_1 = \epsilon - p_{\text{prior}}(\epsilon)$ 。算法 2 给出了伪代码。尽管一步采样是本文的主要关注点，但我们强调，根据该方程，少步采样同样直接可行。

与先前工作的关系。 虽然与先前的一步生成模型 [46, 43, 15, 31, 49, 23, 13, 52] 相关，但我们的方法提供了一个更具原则性的框架。我们方法的核心是

Algorithm 1 MeanFlow: Training.

Note: in PyTorch and JAX, `jvp` returns the function output and JVP.

```
# fn(z, r, t): function to predict u
# x: training batch

t, r = sample_t_r()
e = randn_like(x)

z = (1 - t) * x + t * e
v = e - x

u, dudt = jvp(fn, (z, r, t), (v, 0, 1))

u_tgt = v - (t - r) * dudt
error = u - stopgrad(u_tgt)

loss = metric(error)
```

Algorithm 2 MeanFlow: 1-step Sampling

```
e = randn(x_shape)
x = e - fn(e, r=0, t=1)
```

两个底层场 v 与 u 之间的函数关系，这自然导致了 u 必须满足的平均流恒等式（方程（6））。该恒等式不依赖于神经网络的引入。相比之下，先前的工作通常依赖于对神经网络行为施加额外的一致性约束。一致性模型 [46, 43, 15, 31] 专注于锚定在数据侧的路径：在我们的记号中，这对应于对任意 t 固定 $r \equiv 0$ 。因此，一致性模型仅依赖于单个时间变量，与我们的不同。另一方面，捷径模型 [13] 和 IMM[52] 模型依赖于两个时间变量：它们引入了额外的双时间自洽性约束。相比之下，我们的方法完全由平均速度的定义驱动，用于训练的平均流恒等式（方程（6））自然地该定义导出，无需额外假设。

4.2 带引导的平均流

本文方法自然支持无分类器引导（Classifier-Free Guidance, CFG）[18]。我们并未在采样阶段简单套用 CFG（这会使 NFE 翻倍），而是将 CFG 视为底层真实场的一种属性。这种形式化处理使我们既能享受 CFG 的优势，又能在采样时保持 1-NFE 的行为。

真实场。我们构造一个新的真实场 v^{cfg} ：

$$v^{\text{cfg}}(z_t, t \mid \mathbf{c}) \triangleq \omega v(z_t, t \mid \mathbf{c}) + (1 - \omega) v(z_t, t), \quad (13)$$

它是类别条件场与类别无条件场的线性组合：

$$v(z_t, t \mid \mathbf{c}) \triangleq \mathbb{E}_{p_t(v_t \mid z_t, \mathbf{c})}[v_t] \quad \text{and} \quad v(z_t, t) \triangleq \mathbb{E}_{\mathbf{c}}[v(z_t, t \mid \mathbf{c})], \quad (14)$$

其中 v_t 是条件速度 [28]（在此语境下更准确地说，是样本条件速度）。遵循 MeanFlow 的精神，我们引入对应于 v^{cfg} 的平均速度 u^{cfg} 。根据 MeanFlow 恒等式（式（6））， u^{cfg} 满足：

$$u^{\text{cfg}}(z_t, r, t \mid \mathbf{c}) = v^{\text{cfg}}(z_t, t \mid \mathbf{c}) - (t - r) \frac{d}{dt} u^{\text{cfg}}(z_t, r, t \mid \mathbf{c}). \quad (15)$$

同样， v^{cfg} 和 u^{cfg} 是不依赖于神经网络的底层真实场。此处，式（13）中定义的 v^{cfg} 可改写为：

$$v^{\text{cfg}}(z_t, t \mid \mathbf{c}) = \omega v(z_t, t \mid \mathbf{c}) + (1 - \omega) u^{\text{cfg}}(z_t, t, t), \quad (16)$$

其中我们利用了关系³： $v(z_t, t) = v^{\text{cfg}}(z_t, t)$ 以及 $v^{\text{cfg}}(z_t, t) = u^{\text{cfg}}(z_t, t, t)$ 。

带引导的训练。利用式（15）和式（16），我们构建了一个网络及其学习目标。我们直接用函数 u^{cfg} 参数化 u_{θ}^{cfg} 。基于式（15），我们得到目标：

$$\mathcal{L}(\theta) = \mathbb{E} \|u_{\theta}^{\text{cfg}}(z_t, r, t \mid \mathbf{c}) - \text{sg}(u_{\text{tgt}})\|_2^2, \quad (17)$$

$$\text{where } u_{\text{tgt}} = \tilde{v}_t - (t - r)(\tilde{v}_t \partial_z u_{\theta}^{\text{cfg}} + \partial_t u_{\theta}^{\text{cfg}}). \quad (18)$$

该公式与式（9）类似，唯一区别在于其具有修正的 $\tilde{v}_t$ ：

$$\tilde{v}_t \triangleq \omega v_t + (1 - \omega) u_{\theta}^{\text{cfg}}(z_t, t, t), \quad (19)$$

这由式（16）驱动：式（16）中的项 $v(z_t, t \mid \mathbf{c})$ （即边际速度）被（样本）条件速度 v_t 替换，遵循 [28]。若 $\omega = 1$ ，该损失函数退化为式（9）中的无分类器引导情形。

为使式（17）中的网络 u_{θ}^{cfg} 暴露于类别无条件输入，我们以 10% 的概率丢弃类别条件，遵循 [18]。出于类似的动机，我们也可将式（19）中的 $u_{\theta}^{\text{cfg}}(z_t, t, t)$ 暴露于类别无条件与类别条件两种版本：细节见附录 B.1。

带分类器引导的单次函数评估采样。在我们的公式中， u_{θ}^{cfg} 直接建模 u^{cfg} ，即由分类器引导速度 v^{cfg} （Eq. (13)）诱导的平均速度。因此，采样时无需线性组合：我们直接使用 u_{θ}^{cfg} 进行一步采样（见算法 2），仅需单次函数评估。该公式保持了理想的单次函数评估行为。

³Observe that: $v^{\text{cfg}}(z_t, t) \triangleq \mathbb{E}_{\mathbf{c}}[v^{\text{cfg}}(z_t, t \mid \mathbf{c})] = \omega \mathbb{E}_{\mathbf{c}}[v(z_t, t \mid \mathbf{c})] + (1 - \omega) v(z_t, t) = v(z_t, t)$.

4.3 设计决策

损失度量。在式 (9) 中，所考虑的度量是平方 L2 损失。遵循 [46, 43, 15]，我们研究了不同的损失度量。一般而言，我们考虑如下形式的损失函数 $\mathcal{L} = \|\Delta\|_2^{2\gamma}$ ，其中 Δ 为回归误差。可以证明（见 [15]），最小化 $\|\Delta\|_2^{2\gamma}$ 等价于最小化具有“自适应损失权重”的平方 L2 损失 $\|\Delta\|_2^2$ 。细节见附录。在实践中，我们将权重设为 $w = 1/(\|\Delta\|_2^2 + c)$ ，其中 $p = 1 - \gamma$ 和 $c > 0$ (e.g., 10^{-3})。自适应加权损失为 $\text{sg}(w) \cdot \mathcal{L}$ ，其中 $\mathcal{L} = \|\Delta\|_2^2$ 。若 $p = 0.5$ ，则与 [43] 中的伪胡贝尔损失相似。我们在实验中比较了不同的 p 值。

采样时间步 (r, t) 。我们从预定义分布中采样两个时间步 (r, t) 。我们研究了两种类型的分布：(i) 均匀分布 $\mathcal{U}(0, 1)$ ，以及 (ii) 对数正态 (lognorm) 分布 [11]，其中样本首先从正态分布 $\mathcal{N}(\mu, \sigma)$ 中抽取，然后使用逻辑函数映射到 $(0, 1)$ 。给定采样对，我们将较大值赋给 t ，较小值赋给 r 。我们设置一定比例的随机样本满足 $r = t$ 。

条件化 (r, t) 。我们使用位置嵌入 [48] 对时间变量进行编码，然后将它们组合起来作为神经网络的条件输入。我们注意到，尽管场由 $u_\theta(z_t, r, t)$ 参数化，但网络并不需要直接以 (r, t) 为条件。例如，我们可以让网络直接以 $(t, \Delta t)$ 为条件，其中 $\Delta t = t - r$ 。在这种情况下，我们有 $u_\theta(\cdot, r, t) \triangleq \text{net}(\cdot, t, t - r)$ ，其中 net 为网络。JVP 计算始终针对函数 $u_\theta(\cdot, r, t)$ 进行。我们在实验中比较了不同形式的条件化。

5 实验

实验设置。我们在 ImageNet [7] 生成任务上进行了主要实验，分辨率为 256×256 。我们使用 50K 生成图像评估 Fréchet Inception 距离 (FID) [17]。我们考察函数评估次数 (NFE)，并默认研究 1-NFE 生成。遵循 [34, 13, 52]，我们在预训练 VAE 分词器 [37] 的潜空间上实现模型。对于 256×256 图像，分词器产生 $32 \times 32 \times 4$ 的潜空间，作为模型的输入。我们的模型均从头训练。实现细节见附录 A。

在消融实验中，我们采用 ViT-B/4 架构（即“Base”尺寸，块大小为 4）[9]，如 [34] 所开发，训练 80 个训练轮次（400K 迭代）。作为参考，[34] 中的 DiT-B/4 的 FID 为 68.4，SiT-B/4 [33]（我们的复现）的 FID 为 58.9，均使用 250-NFE 采样。

5.1 消融实验

我们在表 1 中研究模型性质，分析如下：

从流匹配到平均流。我们的方法可视为具有修改目标的流匹配（算法 1），当 r 始终等于 t 时退化为标准流匹配。表 1a 比较了随机采样 $r \neq t$ 的比率。0% 的 $r \neq t$ 比率（退化为流匹配）无法在 1-NFE 生成下产生合理结果。非零的 $r \neq t$ 比率使 MeanFlow 生效，在 1-NFE 生成下产生有意义的结果。我们观察到模型在通过修改目标学习瞬时速度（ $r = t$ ）vs，传播到 $r \neq t$ ）之间取得平衡。此处，最优 FID 在 25% 比率下取得，100% 比率也产生有效结果。

雅可比向量积计算。雅可比向量积运算（式 (8)）是连接所有 (r, t) 坐标的核心关系。在表 1b 中，我们进行了一项破坏性比较，其中故意执行了错误的雅可比向量积计算。结果表明，只有当雅可比向量积计算正确时，才能获得有意义的结果。值得注意的是，沿 $\partial_z u$ 的雅可比向量积切向量是 d 维的，其中 d 是数据维度（此处为 $32 \times 32 \times 4$ ），而沿 $\partial_r u$ 和 $\partial_t u$ 的切向量是一维的。然而，这两个时间变量决定了场 u ，因此尽管它们只是一维的，其作用却至关重要。

条件化 (r, t) 。如第 4.3 节所述，我们可以通过多种形式的显式位置嵌入来表示 $u_\theta(z, r, t)$ ，e.g., $u_\theta(\cdot, r, t) \triangleq \text{net}(\cdot, t, t - r)$ 。表 1c 比较了这些变体。表 1c 显示，所研究的所有 (r, t) 嵌入变体都产生了有意义的 1-NFE 结果，证明了

% of $r \neq t$	FID, 1-NFE	jvp tangent	FID, 1-NFE	pos. embed	FID, 1-NFE
0% (= FM)	328.91	$(v, 0, 1)$	61.06	(t, r)	61.75
25%	61.06	$(v, 0, 0)$	268.06	$(t, t-r)$	61.06
50%	63.14	$(v, 1, 0)$	329.22	$(t, r, t-r)$	63.98
100%	67.32	$(v, 1, 1)$	137.96	$t-r$ only	63.13

(a) 采样 $r \neq t$ 的比率。0% 条目退化为标准流匹配基线。

(b) JVP computation. 正确雅可比量是 $(v, 0, 1)$ ，用于雅可比矩阵 $(\partial_z u, \partial_r u, \partial_t u)$ 。

(c) 位置嵌入。网络以应用于指定变量的嵌入为条件。

t, r sampler	FID, 1-NFE	p	FID, 1-NFE	ω	FID, 1-NFE
uniform(0, 1)	65.90	0.0	79.75	1.0 (w/o cfg)	61.06
lognorm(-0.2, 1.0)	63.83	0.5	63.98	1.5	33.33
lognorm(-0.2, 1.2)	64.72	1.0	61.06	2.0	20.15
lognorm(-0.4, 1.0)	61.06	1.5	66.57	3.0	15.53
lognorm(-0.4, 1.2)	61.79	2.0	69.19	5.0	20.75

(d) Time samplers. t and r are sampled from the specific sampler.

(e) Loss metrics. $p=0$ is squared L2 loss. $p=0.5$ is Pseudo-Huber loss.

(f) CFG scale. Our method supports 1-NFE CFG sampling.

表 1: 1-NFE ImageNet 256 × 256 生成的消融实验。评估 FID-50K。默认配置以灰色标记：B/4 骨干网络，从头训练 80 轮。

证明了 MeanFlow 作为一个框架的有效性。嵌入 $(t, t-r)$ （即时间和区间）取得了最佳结果，而直接嵌入 (r, t) 的表现几乎同样出色。值得注意的是，即使仅嵌入区间 $t-r$ 也能产生合理的结果。

时间采样器。先前工作 [11] 表明，用于采样 t 的分布会影响生成质量。我们在表 1d 中研究了用于采样 (r, t) 的分布。注意， (r, t) 首先被独立采样，随后通过交换进行后处理以强制 $t > r$ ，并将 $r \neq t$ 的比例限制在指定比率。表 1d 报告，对数正态采样器表现最佳，与流匹配 [11] 上的观察一致。

损失度量。已有报告 [43] 指出，损失度量的选择对少步/单步生成的性能有显著影响。我们在表 1e 中研究了这一方面。我们的损失度量通过自适应损失加权 [15] 实现，幂次为 p （第 4.3 节）。表 1e 显示， $p=1$ 取得了最佳结果，而 $p=0.5$ （类似于伪胡贝尔损失 [43]）也表现具有竞争力。标准的平方 L2 损失（此处为 $p=0$ ）相比其他设置表现不佳，但仍能产生有意义的结果，与 [43] 中的观察一致。

引导尺度。表 1f 报告了使用无分类器引导（CFG）的结果。与多步生成 [34] 中的观察一致，CFG 在我们的 1 次函数评估（1-NFE）设置中也显著提升了生成质量。我们强调，我们的 CFG 公式（第 4.2 节）自然支持 1-NFE 采样。

可扩展性。图 4 展示了 MeanFlow 在更大模型规模 and 不同训练时长下的 1-NFE FID 结果。与基于变换器的扩散/流模型（DiT[34] 和 SiT[33]）的行为一致，MeanFlow 模型在 1-NFE 生成中展现出有前景的可扩展性。

5.2 与先前工作的比较

ImageNet 256 × 256 比较。在图 1 中，我们与以往的单步扩散/流模型进行比较，这些模型也总结在表 2（左）中。总体而言，MeanFlow 在其类别中大幅超越了以往的方法：它达到了 **3.43** 的 FID，相对于 IMM 的单步结果 7.77 [52] 有超过 50% 的相对改进；如果我们只比较 1-NFE（不仅仅是单步）生成，MeanFlow 相对于之前的最先进水平（10.60，Shortcut [13]）有近 70% 的相对改进。我们的方法在很大程度上缩小了单步与多步扩散/流模型之间的差距。

在 2-NFE 生成中，我们的方法达到了 **2.20** 的 FID（表 2，左下）。这一结果与多步扩散/流模型的领先基线相当，即 DiT [34]（FID 2.27）和 SiT [33]（FID 2.15），两者在相同的 XL/2 骨干下 NFE 均为 250×2 （表 2，右）。我们的

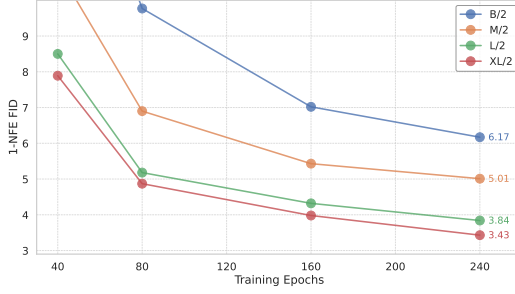


图 4: MeanFlow 模型在 ImageNet 256 $\times$ 256 上的可扩展性。报告了 1-NFE 生成的 FID。所有模型均从头开始训练。在保持 1-NFE 采样行为的同时应用了 CFG。我们的方法在模型规模方面表现出有前景的可扩展性。

method	params	NFE	FID
1-NFE diffusion/flow from scratch			
iCT-XL/2 [43] [†]	675M	1	34.24
Shortcut-XL/2 [13]	675M	1	10.60
MeanFlow-B/2	131M	1	6.17
MeanFlow-M/2	308M	1	5.01
MeanFlow-L/2	459M	1	3.84
MeanFlow-XL/2	676M	1	3.43
2-NFE diffusion/flow from scratch			
iCT-XL/2 [43] [†]	675M	2	20.30
iMM-XL/2 [52]	675M	1 $\times$ 2	7.77
MeanFlow-XL/2	676M	2	2.93
MeanFlow-XL/2+	676M	2	2.20

method	params	NFE	FID
GANs			
BigGAN [5]	112M	1	6.95
GigaGAN [21]	569M	1	3.45
StyleGAN-XL [40]	166M	1	2.30
autoregressive/masking			
AR w/ VQGAN [10]	227M	1024	26.52
MaskGIT [6]	227M	8	6.18
VAR-d30 [47]	2B	10 $\times$ 2	1.92
MAR-H [27]	943M	256 $\times$ 2	1.55
diffusion/flow			
ADM [8]	554M	250 $\times$ 2	10.94
LDM-4-G [37]	400M	250 $\times$ 2	3.60
SimDiff [20]	2B	512 $\times$ 2	2.77
DiT-XL/2 [34]	675M	250 $\times$ 2	2.27
SiT-XL/2 [33]	675M	250 $\times$ 2	2.06
SiT-XL/2+REPA [51]	675M	250 $\times$ 2	1.42

表 2: ImageNet-256 $\times$ 256 上的类别条件生成。所有条目在适用时均报告了 CFG。左: 从头开始训练的 1-NFE 和 2-NFE 扩散/流模型。右: 其他生成模型家族作为参考。在两个表中, “ $\times$ 2” 表示 CFG 在每个采样步骤中产生 2 的 NFE。我们的 MeanFlow 模型均训练了 240 个训练轮次, 除了 “MeanFlow-XL+” 训练了更多轮次, 并采用了为更长训练选择的配置, 详见附录。†: iCT [43] 的结果由 [52] 报告。

结果表明, 少步扩散/流模型可以与其多步前身相媲美。正交改进, 如 REPA [51], 是适用的, 留待未来工作。

值得注意的是, 我们的方法是自包含的, 完全从头开始训练。它在不使用任何预训练、蒸馏或 [43, 15, 31] 中采用的课程学习的情况下, 取得了强劲的结果。

CIFAR-10 比较。 我们在表 3 中报告了 CIFAR-10 [25] (32×32) 上的无条件生成结果。FID-50K 以 1-NFE 采样报告。所有条目均使用相同的 U-net [38], 该网络源自 [44] (~ 55 M), 直接应用于像素空间。所有其他竞争者均采用 EDM 风格预条件器 [22], 而我们的方法没有预条件器。实现细节见附录。在该数据集上, 我们的方法与先前方法具有竞争力。

method	precond	NFE	FID
iCT [43]	EDM	1	2.83
ECT [15]	EDM	1	3.60
sCT [31]	EDM	1	2.97
IMM [52]	EDM	1	3.20
MeanFlow	none	1	2.92

表 3: 无条件 CIFAR-10。

6 结论

我们提出了 MeanFlow, 一个用于一步生成的原则性且有效的框架。广义而言, 本文所考虑的场景与物理学中的多尺度仿真问题相关, 这些问题可能涉及空间或时间上的一系列尺度、长度和分辨率。进行数值仿真本质上受限于计算机解析尺度范围的能力。我们的公式涉及在粗粒度级别描述底层量, 这是物理学中许多重要应用的共同主题。我们希望我们的工作能够桥接生成建模、仿真及相关领域动力系统的研究。

致谢

我们非常感谢谷歌 TPU 研究云 (TRC) 授予我们 TPU 的使用权限。Zhengyang Geng 部分由博世人工智能中心资助。Zico Kolter 衷心感谢博世对实验室的资助。Mingyang Deng 和 Xingjian Bai 部分由 MIT-IBM Watson AI 实验室资助奖项支持。我们感谢 Runqian Wang、Qiao Sun、Zhicheng Jiang、Hanhong Zhao、Yiyang Lu 和 Xianbang Wang 在 JAX 和 TPU 实现上的帮助。

参考文献

- [1] Michael S Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. *arXiv preprint arXiv:2209.15571*, 2022. 2
- [2] Michael Samuel Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *International Conference on Learning Representations (ICLR)*, 2023. 1, 2
- [3] Nicholas M Boffi, Michael S Albergo, and Eric Vanden-Eijnden. Flow map matching. *arXiv preprint arXiv:2406.07507*, 2024. 2
- [4] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/google/jax>. 16
- [5] Andrew Brock, Jeff Donahue, and Karen Simonyan. Large scale GAN training for high fidelity natural image synthesis. In *International Conference on Learning Representations (ICLR)*, 2019. 9
- [6] Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T Freeman. Maskgit: Masked generative image transformer. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 9
- [7] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009. 2, 7
- [8] Prafulla Dhariwal and Alexander Nichol. Diffusion models beat gans on image synthesis. *Neural Information Processing Systems (NeurIPS)*, 34, 2021. 9
- [9] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021. 7, 14
- [10] Patrick Esser, Robin Rombach, and Bjorn Ommer. Taming transformers for high-resolution image synthesis. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. 9
- [11] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first international conference on machine learning*, 2024. 1, 7, 8
- [12] Tor Fjelde, Emile Mathieu, and Vincent Dutordoir. An introduction to flow matching. <https://mlg.eng.cam.ac.uk/blog/2024/01/20/flow-matching.html>, January 2024. Cambridge Machine Learning Group Blog. 3
- [13] Kevin Frans, Danijar Hafner, Sergey Levine, and Pieter Abbeel. One step diffusion via shortcut models. In *International Conference on Learning Representations (ICLR)*, 2025. 1, 2, 5, 6, 7, 8, 9

- [14] Zhengyang Geng, Ashwini Pokle, and J Zico Kolter. One-step diffusion distillation via deep equilibrium models. *Neural Information Processing Systems (NeurIPS)*, 36, 2024. 2
- [15] Zhengyang Geng, Ashwini Pokle, William Luo, Justin Lin, and J Zico Kolter. Consistency models made easy. *arXiv preprint arXiv:2406.14548*, 2024. 1, 2, 5, 6, 7, 8, 9, 15
- [16] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch sgd: Training imagenet in 1 hour. *arXiv preprint arXiv:1706.02677*, 2017. 14
- [17] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. *Neural Information Processing Systems (NeurIPS)*, 2017. 7
- [18] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022. 2, 6, 14, 15
- [19] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Neural Information Processing Systems (NeurIPS)*, 2020. 1, 2
- [20] Emiel Hoogeboom, Jonathan Heek, and Tim Salimans. simple diffusion: End-to-end diffusion for high resolution images. In *International Conference on Machine Learning (ICML)*, 2023. 9
- [21] Minguk Kang, Jun-Yan Zhu, Richard Zhang, Jaesik Park, Eli Shechtman, Sylvain Paris, and Taesung Park. Scaling up gans for text-to-image synthesis. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 9
- [22] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. In *Neural Information Processing Systems (NeurIPS)*, 2022. 2, 9, 14
- [23] Dongjun Kim, Chieh-Hsin Lai, Wei-Hsiang Liao, Naoki Murata, Yuhta Takida, Toshimitsu Uesaka, Yutong He, Yuki Mitsufuji, and Stefano Ermon. Consistency trajectory models: Learning probability flow ODE trajectory of diffusion. In *International Conference on Learning Representations (ICLR)*, 2024. 5
- [24] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. 14
- [25] Alex Krizhevsky. Learning multiple layers of features from tiny images. 2009. URL <https://www.cs.toronto.edu/~kriz/cifar.html>. 9
- [26] Gottfried Wilhelm Leibniz. Epistola LXXI ad johannem bernoullium, 5 aug 1697. In Johann Bernoulli, editor, *Virorum celeberrimorum G. G. Leibnitii et Johannis Bernoullii commercium philosophicum et mathematicum*, volume I, pages 368–370. Marc-Michel Bousquet, Lausanne & Geneva, 1745. URL https://archive.org/details/bub_gb_l03wOMxjoF8C/page/368. First explicit statement of the Leibniz integral rule. 4
- [27] Tianhong Li, Yonglong Tian, He Li, Mingyang Deng, and Kaiming He. Autoregressive image generation without vector quantization. *Neural Information Processing Systems (NeurIPS)*, 2024. 9
- [28] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023. 1, 2, 3, 5, 6
- [29] Yaron Lipman, Marton Havasi, Peter Holderrieth, Neta Shaul, Matt Le, Brian Karrer, Ricky T. Q. Chen, David Lopez-Paz, Heli Ben-Hamu, and Itai Gat. Flow matching guide and code, 2024. URL <https://arxiv.org/abs/2412.06264>. 14
- [30] Xingchao Liu, Chengyue Gong, and qiang liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023. 1, 2, 3

- [31] Cheng Lu and Yang Song. Simplifying, stabilizing and scaling continuous-time consistency models. In *International Conference on Learning Representations (ICLR)*, 2025. 1, 2, 5, 6, 9
- [32] Weijian Luo, Tianyang Hu, Shifeng Zhang, Jiacheng Sun, Zhenguo Li, and Zhihua Zhang. Diff-instruct: A universal approach for transferring knowledge from pre-trained diffusion models. *Neural Information Processing Systems (NeurIPS)*, 2024. 2
- [33] Nanye Ma, Mark Goldstein, Michael S Albergo, Nicholas M Boffi, Eric Vanden-Eijnden, and Saining Xie. Sit: Exploring flow and diffusion-based generative models with scalable interpolant transformers. In *European Conference on Computer Vision (ECCV)*, 2024. 1, 7, 8, 9
- [34] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 7, 8, 9, 14
- [35] Adam Polyak, , et al. Movie Gen: A cast of media foundation models, 2025. 1
- [36] Danilo Rezende and Shakir Mohamed. Variational inference with normalizing flows. In *International Conference on Machine Learning (ICML)*, 2015. 2
- [37] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. 7, 9
- [38] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention (MICCAI)*, 2015. 9, 14
- [39] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In *International Conference on Learning Representations (ICLR)*, 2022. 2
- [40] Axel Sauer, Katja Schwarz, and Andreas Geiger. Stylegan-xl: Scaling stylegan to large diverse datasets. In *ACM Transactions on Graphics (SIGGRAPH)*, 2022. 9
- [41] Axel Sauer, Dominik Lorenz, Andreas Blattmann, and Robin Rombach. Adversarial diffusion distillation. In *European Conference on Computer Vision (ECCV)*, 2024. 2
- [42] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning (ICML)*, 2015. 1, 2
- [43] Yang Song and Prafulla Dhariwal. Improved techniques for training consistency models. In *International Conference on Learning Representations (ICLR)*, 2024. 1, 2, 5, 6, 7, 8, 9, 15
- [44] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. *Neural Information Processing Systems (NeurIPS)*, 2019. 1, 2, 9, 14
- [45] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2021. 2
- [46] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *International Conference on Machine Learning (ICML)*, 2023. 1, 2, 3, 5, 6, 7
- [47] Keyu Tian, Yi Jiang, Zehuan Yuan, Bingyue Peng, and Liwei Wang. Visual autoregressive modeling: Scalable image generation via next-scale prediction. *Neural Information Processing Systems (NeurIPS)*, 2024. 9
- [48] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Neural Information Processing Systems (NeurIPS)*, 2017. 7
- [49] Ling Yang, Zixiang Zhang, Zhilong Zhang, Xingchao Liu, Minkai Xu, Wentao Zhang, Chenlin Meng, Stefano Ermon, and Bin Cui. Consistency flow matching: Defining straight flows with velocity consistency. *arXiv preprint arXiv:2407.02398*, 2024. 2, 5

- [50] Tianwei Yin, Michaël Gharbi, Richard Zhang, Eli Shechtman, Frédo Durand, William T Freeman, and Taesung Park. One-step diffusion with distribution matching distillation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2
- [51] Sihyun Yu, Sangkyung Kwak, Huiwon Jang, Jongheon Jeong, Jonathan Huang, Jinwoo Shin, and Saining Xie. Representation alignment for generation: Training diffusion transformers is easier than you think. In *International Conference on Learning Representations (ICLR)*, 2025. 9
- [52] Linqi Zhou, Stefano Ermon, and Jiaming Song. Inductive moment matching. *arXiv preprint arXiv:2503.07565*, 2025. 1, 2, 5, 6, 7, 8, 9
- [53] Mingyuan Zhou, Huangjie Zheng, Zhendong Wang, Mingzhang Yin, and Hai Huang. Score identity distillation: Exponentially fast distillation of pretrained diffusion models for one-step generation. In *International Conference on Machine Learning (ICML)*, 2024. 2

附录

A 实现

表 4: ImageNet 256 × 256. 上的配置 B/4 是我们的消融模型。

configs	B/4	B/2	M/2	L/2	XL/2	XL/2+
params (M)	131	131	497.8	459	676	676
FLOPs (G)	5.6	23.1	54.0	119.0	119.0	119.0
depth	12	12	16	24	28	28
hidden dim	768	768	1024	1024	1152	1152
heads	12	12	16	16	16	16
patch size	4×4	2×2	2×2	2×2	2×2	2×2
epochs	80	240	240	240	240	1000
batch size			256			
dropout			0.0			
optimizer			Adam [24]			
lr schedule			constant			
lr			0.0001			
Adam (β_1, β_2)			(0.9, 0.95)			
weight decay			0.0			
ema decay			0.9999			
ratio of $r \neq t$	Tab. 1a			25%		
(r, t) cond	Tab. 1c			$(t, t - r)$		
(r, t) sampler	Tab. 1d			lognorm(-0.4, 1.0)		
p for adaptive weight	Tab. 1e			1.0		
CFG effective scale ω'	Tab. 1f	2.0	2.0	2.5	2.5	2.0
CFG ω , Eq. (21)	$\omega = \omega'$	1.0	1.0	0.2	0.2	1.0
CFG κ , Eq. (21)			$\kappa = 1 - \omega/\omega'$			
CFG 类别条件丢弃 [18]			0.1 [18]			
若 t 在以下情况中则触发 CFG:	[0.0, 1.0]	[0.0, 1.0]	[0.0, 1.0]	[0.0, 0.8]	[0.0, 0.75]	[0.3, 0.8]

ImageNet 256 × 256. 我们使用标准的 VAE 分词器提取潜在表示。⁴ 潜在尺寸为 $32 \times 32 \times 4$ ，作为模型的输入。骨干架构遵循 DiT [34]，基于 ViT [9] 并采用 adaLN-Zero [34] 进行条件化。为了对两个时间变量（*e.g.*, (r, t) ）进行条件化，我们对每个时间变量应用位置嵌入，随后通过一个 2 层 MLP，并求和。我们保持 DiT 架构块不变，而架构改进是正交且可行的。具体配置见表 4。

CIFAR-10. 我们在 CIFAR-10 上实验类别无条件生成。我们的实现遵循标准的流匹配实践 [29]。模型的输入是像素空间中的 $32 \times 32 \times 3$ 。网络是一个 U-net [38]，由 [44] (~ 55 M) 发展而来，这是其他基线常用的结构。我们对两个时间变量（此处为 $(t, t - r)$ ）应用位置嵌入，并将它们拼接用于条件化。我们不使用任何 EDM 预处理器 [22]。

我们使用 Adam 优化器，学习率为 0.0006，批大小为 1024， $(\beta_1, \beta_2) = (0.9, 0.999)$ ，随机失活率为 0.2，权重衰减为 0，EMA 衰减率为 0.99995。模型训练 800K 次迭代（含 10K 预热 [16]）。 (r, t) 采样器为 lognorm(-2.0, 2.0)。采样 $r \neq t$ 的比例为 75%。自适应加权的幂 p 为 0.75。我们的数据增强设置遵循 [22]，禁用垂直翻转和旋转。

⁴<https://huggingface.co/pcuenq/sd-vae-ft-mse-flax>

κ	FID, 1-NFE
0.0	20.15
0.5	19.15
0.8	19.10
0.9	18.63
0.95	19.17

表 5: MeanFlow 的改进 CFG . κ 如式 (20) 所定义, 其目标是使类别条件 $u^{\text{cfg}}(\cdot | \mathbf{c})$ 和类别无条件 $u^{\text{cfg}}(\cdot)$ 都能出现在目标中。在本表中, 我们

固定有效引导尺度 ω' (由 $\omega' = \omega/(1-\kappa)$ 给出) 为 2.0。相应地, 对于不同的 κ 值, 我们通过 $\omega = (1-\kappa) \cdot \omega'$ 设置 ω 。若 $\kappa = 0$, 则退化为式 (19) 中的 CFG 情况 (另见表 1f)。与标准 CFG 随机丢弃类别条件的做法 [18] 类似, 我们观察到在目标中混合类别条件和类别无条件 u^{cfg} 可提升生成质量。

B 补充技术细节

B.1 改进的无分类器引导 (CFG) 用于平均流 (MeanFlow)

在第 4.2 节中, 我们讨论了如何自然地将我们的方法扩展到支持无分类器引导 (CFG)。唯一需要的改动是根据公式 (19) 修改目标。我们注意到公式 (19) 中只出现了类别无条件项 u^{cfg} 。在原始的无分类器引导 [18] 中, 通常的做法是通过随机丢弃来混合类别条件和类别无条件预测。我们观察到类似的想法也可以应用于我们的回归目标。

形式上, 我们引入一个混合尺度 κ , 并将公式 (16) 重写为:

$$v^{\text{cfg}}(z_t, t | \mathbf{c}) = \omega v(z_t, t | \mathbf{c}) + \kappa u^{\text{cfg}}(z_t, t, t | \mathbf{c}) + (1-\omega-\kappa) u^{\text{cfg}}(z_t, t, t). \quad (20)$$

这里, κ 的作用是在右侧与 $u^{\text{cfg}}(\cdot | \mathbf{c})$ 进行混合。我们可以证明公式 (20) 满足原始的无分类器引导公式 (公式 (13)), 其有效引导尺度为 $\omega' = \frac{\omega}{1-\kappa}$, 利用了我们在推导公式 (16) 时使用的关系 $v(z_t, t) = v^{\text{cfg}}(z_t, t)$ 和 $v^{\text{cfg}}(z_t, t) = u^{\text{cfg}}(z_t, t, t)$ 。由此, 公式 (19) 被重写为

$$\tilde{v}_t \triangleq \underbrace{\omega(\epsilon - x)}_{\text{sample } v_t} + \underbrace{\kappa u_{\theta}^{\text{cfg}}(z_t, t, t | \mathbf{c})}_{\text{cls-cond output}} + \underbrace{(1-\omega-\kappa) u_{\theta}^{\text{cfg}}(z_t, t, t)}_{\text{cls-uncond output}}. \quad (21)$$

损失函数与公式 (17) 中定义的相同。

引入 κ 的影响在表 5 中进行了探索, 其中我们将有效引导尺度 ω' 固定为 2.0, 并改变 κ 。结果表明, 通过 κ 进行混合可以进一步提高生成质量。我们注意到, 主论文表 1f 中的消融实验并未涉及这一改进, *i.e.*, κ 被设置为 0。

B.2 损失度量

平方 L2 损失由下式给出 $\mathcal{L} = \|\Delta\|_2^2$, where $\Delta = u_{\theta} - u_{\text{tgt}}$ denotes the regression error.

Generally, one can adopt the 2γ loss $\mathcal{L}_{\gamma} = \|\Delta\|_2^{2\gamma}$, where γ is a user-specified hyper-parameter. Minimizing this loss is equivalent to minimizing an adaptively weighted squared L2 loss (see [15]): $\frac{d}{d\theta} \mathcal{L}_{\gamma} = \gamma(\|\Delta\|_2^2)^{\gamma-1} \cdot \frac{d\|\Delta\|_2^2}{d\theta}$. This can be viewed as weighting the squared L2 loss ($\|\Delta\|_2^2$) by a loss-adaptive weight $\lambda \propto \|\Delta\|_2^{2(\gamma-1)}$. In practice, we follow [15] and weight by:

$$w = 1/(\|\Delta\|_2^2 + c)^p, \quad (22)$$

其中 $p = 1 - \gamma$ 和 $c > 0$ 是一个避免除零的小常数。如果 $p = 0.5$, 这与 [43] 中的伪胡贝尔损失相似。自适应加权损失为 $\text{sg}(w) \cdot \mathcal{L}$, 其中 sg 表示停止梯度算子。

B.3 关于平均流恒等式充分性的说明

在主论文中, 从定义式 (4) 出发, 我们推导出了平均流恒等式 (6)。这表明“式 (4) $\Rightarrow$ 式 (6)”, 即式 (6) 是式 (4) 的必要条件。接下来, 我们证明它也是充分条件, 即“式 (6) $\Rightarrow$ 式 (4)”。一般来说, 导数相等并不意味着积分相等: 它们可能相差一个常数。在我们的情况中, 我们证明该常数会被抵消。考虑一个“位移场” S , 写为:

$$S(z_t, r, t) = (t - r)u(z_t, r, t). \quad (23)$$



图 5: 1-NFE 生成结果。我们展示了使用我们的 1-NFE 模型 (MeanFlow-XL/2, 3.43 FID) 在 ImageNet 256×256 上进行类别条件生成的精选示例。

如果我们将 S 视为任意函数，那么一般来说，导数相等只能导致积分相等，最多相差某个常数：

$$\frac{d}{dt}S(z_t, r, t) = v(z_t, t) \implies S(z_t, r, t) + C_1 = \int_r^t v(z_\tau, \tau) d\tau + C_2. \quad (24)$$

然而， S 的定义给出 $S|_{t=r} = 0$ ，并且当 $t = r$ 时我们还有 $\int_r^t v d\tau = 0$ ，这给出 $C_1 = C_2$ 。这表明“式 (6) $\Rightarrow$ 式 (4)”。

我们注意到，这种充分性是对平均速度 u 建模的结果，而不是直接对位移 S 建模。例如，对位移施加导数相等，即 $\frac{d}{dt}S = v$ ，并不会自动得到 $S = \int_r^t v d\tau$ 。如果我们直接参数化 S ，则需要额外的边界条件 $S|_{t=r} = 0$ 。我们的公式可以自动满足这一条件。

B.4 雅可比向量积 (JVP) 计算分析

虽然在某些方法中，雅可比向量积 (JVP) 计算可能被视为一个关注点，但在我们的方法中它非常轻量。在我们的情况下，由于计算得到的乘积（雅可比矩阵与切向量之间）受到停止梯度 (stop-gradient) 约束（公式 (10)），在使用反向传播进行随机梯度下降 (SGD) 时，它被视为 *w.r.t.* θ 常数。因此，它不会给 θ -反向传播增加任何开销。

因此，计算该乘积所产生的额外计算是 jvp 操作执行的“反向”传递。这种反向传递类似于用于优化神经网络的标准反向传递 (*w.r.t.* θ)——事实上，在一些深度学习库中，如 JAX [4]，它们使用类似的接口来计算标准反向传播 (*w.r.t.* θ)。在我们的情况下，它甚至比标准反向传播成本更低，因为它只需要反向传播到输入变量，而不需要到参数 θ 。这种开销很小。

我们在消融模型 B/4 中对 JVP 引起的开销进行了基准测试。我们在 JAX 中实现该模型，并在 v4-8 TPU 上进行基准测试。我们将 MeanFlow 与其流匹配 (Flow Matching) 对应方法进行比较，其中唯一的开销是 JVP 引起的反向传递；注意，JVP 的前向传递是标准的前向传播，流匹配（或任何典型的目标函数）也需要执行这一过程。在我们的基准测试中，使用流匹配的训练成本为 0.045 秒/迭代，使用 MeanFlow 为 0.052 秒/迭代，这仅仅是 16% 的墙钟时间开销。

C 定性结果

图 5 展示了使用我们的 1-NFE 模型在 ImageNet 256×256 上生成的精选示例。